

Exercícios — Polimorfismo, Interfaces e Injeção de Dependência

Cinco exercícios novos, em contextos diferentes, seguindo o mesmo padrão do exercício de Employee: persistir uma entidade de domínio criando uma interface Repository genérica, um serviço de registro, um serviço de busca e uma implementação concreta em memória.

Exercício 1 — Student

```
Student
- id: String
- name: String
- course: String
- gpa: double
- enrollmentDate: LocalDate
+ getYearsEnrolled(): double
+ isEligibleForGraduation(): boolean
```

Você precisa persistir a classe Student descrita no diagrama acima (getYearsEnrolled() calcula os anos desde enrollmentDate até hoje; isEligibleForGraduation() retorna true se getYearsEnrolled() for maior ou igual a 4 e gpa for maior ou igual a 6.0). Para isso, crie as seguintes classes:

- ❖ Repository: interface genérica parametrizável quanto ao tipo da chave e da entidade persistida. A interface deve prover um método para salvar uma entidade e outro para obter a entidade pelo id.
- ❖ RegisterStudentService: classe que recebe uma instância de repositório por injeção de dependência e fornece um método register(Student s), que persiste o student se ele já não estiver registrado no sistema.
- ❖ FindStudentService: classe que recebe uma instância de repositório por injeção de dependência e fornece um método findById(String id), que retorna um student que contenha o id, se ele estiver registrado no sistema.
- ❖ FakeStudentRepository: classe concreta e não genérica que implemente a interface Repository para persistir students na memória utilizando um array.

Exercício 2 — Client

```
Client
- id: String
- name: String
- email: String
- loyaltyPoints: int
- registrationDate: LocalDate
+ getMembershipDuration(): double
+ calculateDiscount(): double
```

Você precisa persistir a classe Client descrita no diagrama acima (getMembershipDuration() calcula os anos desde registrationDate até hoje; calculateDiscount() retorna um percentual de desconto igual a 1% por ano de associação, mais 0,01% por ponto de fidelidade acumulado). Para isso, crie as seguintes classes:

- ❖ Repository: interface genérica parametrizável quanto ao tipo da chave e da entidade persistida. A interface deve prover um método para salvar uma entidade e outro para obter a entidade pelo id.
- ❖ RegisterClientService: classe que recebe uma instância de repositório por injeção de dependência e fornece um método register(Client c), que persiste o client se ele já não estiver registrado no sistema.

- ❖ FindClientService: classe que recebe uma instância de repositório por injeção de dependência e fornece um método findById(String id), que retorna um client que contenha o id, se ele estiver registrado no sistema.
- ❖ FakeClientRepository: classe concreta e não genérica que implemente a interface Repository para persistir clients na memória utilizando uma ArrayList (em vez de array).

Exercício 3 — Product

```
Product
- id: int
- name: String
- price: double
- stockQuantity: int
- manufactureDate: LocalDate
+ getShelfLifeInDays(): long
+ applyPriceAdjustment(percentage: double): void
```

Você precisa persistir a classe Product descrita no diagrama acima (getShelfLifeInDays() calcula quantos dias se passaram desde manufactureDate até hoje; applyPriceAdjustment(percentage) aumenta ou reduz o atributo price de acordo com o percentual informado, que pode ser positivo ou negativo). Note que, diferentemente dos exemplos anteriores, a chave primária de Product é do tipo int, e não String. Para isso, crie as seguintes classes:

- ❖ Repository: interface genérica parametrizável quanto ao tipo da chave e da entidade persistida. A interface deve prover um método para salvar uma entidade e outro para obter a entidade pelo id.
- ❖ RegisterProductService: classe que recebe uma instância de repositório por injeção de dependência e fornece um método register(Product p), que persiste o product se ele já não estiver registrado no sistema.
- ❖ FindProductService: classe que recebe uma instância de repositório por injeção de dependência e fornece um método findById(int id), que retorna um product que contenha o id, se ele estiver registrado no sistema.
- ❖ FakeProductRepository: classe concreta e não genérica que implemente a interface Repository<Integer, Product> para persistir products na memória utilizando um array.

Exercício 4 — Vehicle

```
Vehicle
- id: String
- plate: String
- model: String
- mileage: double
- purchaseDate: LocalDate
+ getAge(): double
+ calculateMaintenanceCost(): double
```

Você precisa persistir a classe Vehicle descrita no diagrama acima (getAge() calcula os anos desde purchaseDate até hoje; calculateMaintenanceCost() retorna um custo estimado de manutenção, calculado como R\$ 150,00 por ano de idade do veículo somado a R\$ 0,05 por quilômetro rodado registrado em mileage). Para isso, crie as seguintes classes:

- ❖ Repository: interface genérica parametrizável quanto ao tipo da chave e da entidade persistida. A interface deve prover um método para salvar uma entidade, outro para obter a entidade pelo id e um terceiro método boolean existsById(K id), que indica se já existe uma entidade cadastrada com

aquele id.

- ❖ **RegisterVehicleService**: classe que recebe uma instância de repositório por injeção de dependência e fornece um método `register(Vehicle v)`, que persiste o vehicle utilizando o novo método `existsById` para verificar se ele já não estava registrado no sistema.
- ❖ **FindVehicleService**: classe que recebe uma instância de repositório por injeção de dependência e fornece um método `findById(String id)`, que retorna um vehicle que contenha o id, se ele estiver registrado no sistema.
- ❖ **FakeVehicleRepository**: classe concreta e não genérica que implemente a interface `Repository` para persistir vehicles na memória utilizando um array.

Exercício 5 — Book

```
Book
- isbn: String
- title: String
- author: String
- pages: int
- publicationDate: LocalDate
+ getYearsSincePublication(): double
+ calculatePopularityScore(borrowCount: int): double
```

Você precisa persistir a classe `Book` descrita no diagrama acima, usando o atributo `isbn` como chave de identificação (`getYearsSincePublication()` calcula os anos desde `publicationDate` até hoje; `calculatePopularityScore(borrowCount)` retorna um índice de popularidade igual ao número de empréstimos (`borrowCount`) dividido pelo número de anos desde a publicação, somado a 1 para evitar divisão por zero). Para isso, crie as seguintes classes:

- ❖ **Repository**: interface genérica parametrizável quanto ao tipo da chave e da entidade persistida. A interface deve prover um método para salvar uma entidade e outro para obter a entidade pelo id.
- ❖ **RegisterBookService**: classe que recebe uma instância de repositório por injeção de dependência e fornece um método `register(Book b)`, que persiste o book se ele já não estiver registrado no sistema (comparando pelo `isbn`).
- ❖ **FindBookService**: classe que recebe uma instância de repositório por injeção de dependência e fornece um método `findById(String isbn)`, que retorna um book que contenha o `isbn` informado, se ele estiver registrado no sistema.
- ❖ **FakeBookRepository**: classe concreta e não genérica que implemente a interface `Repository` para persistir books na memória. Diferentemente dos exercícios anteriores, esta implementação deve usar um `HashMap<String, Book>` internamente em vez de um array ou `ArrayList`, mantendo o mesmo contrato definido pela interface `Repository`.